

LAPORAN PRAKTIKUM STRUKTUR DATA

“PRATIKUM 8”

Disusun Oleh:

Aryahiyahul Fikra

2511532026

Dosen Pengampu:

Dr. Wahyudi S.T,M.T

Asisten Pembimbing:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Daftar Isi

Daftar Isi.....	2
BAB I	3
PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan Pratikum	3
BAB II.....	4
PEMBAHASAN	4
2.1 BubbleSortGUI.....	4
2.2 Shell Sort.....	6
2.3 Merge Sort.....	7
2.4 Quick Sort.....	8
BAB III.....	9
PENUTUP	9
3.1 Kesimpulan.....	9
3.2 Saran.....	9
TUGAS PEKAN 8	10
Kelas Lagu_2511532026.....	10
Kelas Sorting_2511532026	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data (sorting) merupakan salah satu proses penting dalam pengolahan data. Dengan data yang terurut, proses pencarian, pengelompokan, dan pengolahan informasi dapat dilakukan lebih cepat dan efisien. Dalam pemrograman terdapat berbagai metode sorting yang memiliki karakteristik dan tingkat efisiensi yang berbeda-beda.

Pada praktikum ini dipelajari beberapa algoritma pengurutan yaitu Bubble Sort, Shell Sort, Merge Sort, dan Quick Sort menggunakan bahasa pemrograman Java. Selain itu, Bubble Sort juga diimplementasikan dalam bentuk Graphical User Interface (GUI) sehingga proses pengurutan dapat diamati langkah demi langkah.

1.2 Tujuan Pratikum

Tujuan pratikum ini Adalah:

1. Memahami konsep algoritma pengurutan data.
2. Mengimplementasikan Bubble Sort, Shell Sort, Merge Sort, dan Quick Sort menggunakan Java.
3. Membandingkan cara kerja masing-masing algoritma sorting.
4. Memahami visualisasi proses sorting menggunakan GUI.

BAB II

PEMBAHASAN

2.1 BubbleSortGUI

Bubble Sort merupakan algoritma pengurutan sederhana yang bekerja dengan cara membandingkan dua elemen yang berdekatan, kemudian menukarnya jika posisinya tidak sesuai urutan. Proses ini dilakukan berulang kali hingga seluruh data terurut.

Pada program **BubbleSortGUI_2511532026**, pengguna dapat memasukkan data melalui textbox. Selanjutnya proses sorting dapat dijalankan secara bertahap menggunakan tombol *Langkah Selanjutnya*. Setiap langkah akan ditampilkan pada area log sehingga pengguna dapat melihat proses pertukaran elemen secara visual.

Kelebihan:

1. Mudah dipahami.
2. Cocok untuk pembelajaran algoritma sorting.

Kekurangan:

Kurang efisien untuk jumlah data yang besar.

Kode Program:

```
1 import java.awt.BorderLayout;
2
3 public class BubbleSortGUI_2511532026 extends JFrame {
4
5     private static final long serialVersionUID = 1L;
6     private int[] array;
7     private JLabel[] labelArray;
8
9     private JButton stepButton, resetButton, setButton;
10    private JTextField inputField;
11
12    private JPanel panelArray;
13    private JTextArea stepArea;
14
15    private int i = 0, j = 0;
16    private boolean sorting = false;
17    private int stepCount = 1;
18
19    public BubbleSortGUI_2511532026() {
20
21        setTitle("Bubble Sort Langkah per Langkah");
22        setSize(750, 400);
23        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24        setLocationRelativeTo(null);
25        setLayout(new BorderLayout());
26
27        JPanel inputPanel = new JPanel(new FlowLayout());
28
29        inputField = new JTextField(30);
30        setButton = new JButton("Set Array");
31
32        inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma) : "));
33        inputPanel.add(inputField);
34        inputPanel.add(setButton);
35
36        panelArray = new JPanel();
37        panelArray.setLayout(new FlowLayout());
38
39        JPanel controlPanel = new JPanel();
40
41        stepButton = new JButton("Langkah Selanjutnya");
42        resetButton = new JButton("Reset");
43    }
```

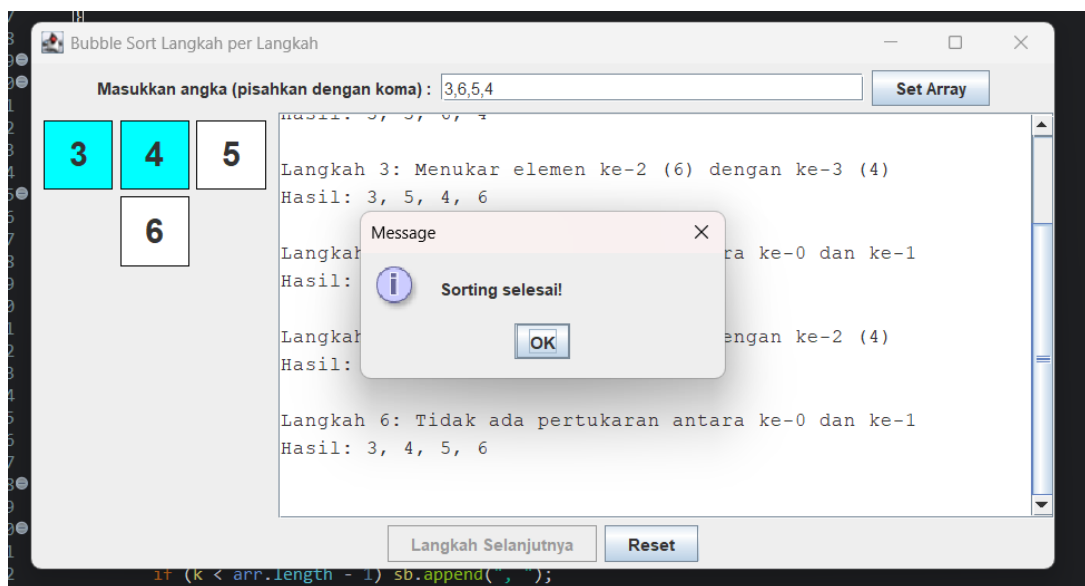
```
44        stepButton.setEnabled(false);
45
46        controlPanel.add(stepButton);
47        controlPanel.add(resetButton);
48
49        stepArea = new JTextArea(8, 60);
50        stepArea.setEditable(false);
51        stepArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
52
53        JScrollPane scrollPane = new JScrollPane(stepArea);
54
55        add(inputPanel, BorderLayout.NORTH);
56        add(panelArray, BorderLayout.CENTER);
57        add(controlPanel, BorderLayout.SOUTH);
58        add(scrollPane, BorderLayout.EAST);
59
60        setButton.addActionListener(e -> setArrayFromInput());
61        stepButton.addActionListener(e -> performStep());
62        resetButton.addActionListener(e -> reset());
63    }
64
65    private void setArrayFromInput() {
66
67        String text = inputField.getText().trim();
68
69        if (text.isEmpty()) return;
70
71        String[] parts = text.split(",");
72        array = new int[parts.length];
73
74        try {
75            for (int k = 0; k < parts.length; k++) {
76                array[k] = Integer.parseInt(parts[k].trim());
77            }
78        } catch (NumberFormatException e) {
79            JOptionPane.showMessageDialog(
80                this,
81                "Masukkan hanya angka " + "yang dipisahkan koma!",
82                "Error",
83                JOptionPane.ERROR_MESSAGE);
84            return;
85        }
86    }
```

```

107     i = 0;
108     j = 0;
109     stepCount = 1;
110     sorting = true;
111     stepButton.setEnabled(true);
112     stepArea.setText("");
113     panelArray.removeAll();
114
115     labelArray = new JLabel[array.length];
116
117     for (int k = 0; k < array.length; k++) {
118         labelArray[k] = new JLabel(String.valueOf(array[k]));
119         labelArray[k].setFont(new Font("Arial", Font.BOLD, 24));
120         labelArray[k].setOpaque(true);
121         labelArray[k].setBackground(Color.WHITE);
122         labelArray[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
123         labelArray[k].setPreferredSize(new Dimension(50, 50));
124         labelArray[k].setHorizontalAlignment(SwingConstants.CENTER);
125         panelArray.add(labelArray[k]);
126     }
127
128     panelArray.revalidate();
129     panelArray.repaint();
130 }
131
132 private void performStep() {
133     if (!sorting || i >= array.length - 1) {
134         sorting = false;
135         stepButton.setEnabled(false);
136         JOptionPane.showMessageDialog(this, "Sorting selesai!");
137         return;
138     }
139
140     resetHighlights();
141
142     StringBuilder stepLog = new StringBuilder();
143
144     labelArray[j].setBackground(Color.CYAN);
145     labelArray[j + 1].setBackground(Color.CYAN);
146
147     if (array[j] > array[j + 1]) {
148         // Swap
149
181     }
182
183     private void updateLabels() {
184         for (int k = 0; k < array.length; k++) {
185             labelArray[k].setText(String.valueOf(array[k]));
186         }
187     }
188
189     private void resetHighlights() {
190         for (JLabel label : labelArray) {
191             label.setBackground(Color.WHITE);
192         }
193     }
194
195     private void reset() {
196         inputField.setText("");
197         panelArray.removeAll();
198         panelArray.revalidate();
199         panelArray.repaint();
200         stepArea.setText("");
201         stepButton.setEnabled(false);
202         sorting = false;
203         i = 0;
204         j = 0;
205         stepCount = 1;
206     }
207
208     private String arrayToString(int[] arr) {
209         StringBuilder sb = new StringBuilder();
210         for (int k = 0; k < arr.length; k++) {
211             sb.append(arr[k]);
212             if (k < arr.length - 1) sb.append(", ");
213         }
214         return sb.toString();
215     }
216
217     public static void main(String[] args) {
218         SwingUtilities.invokeLater(() -> {
219             BubbleSortGUI_2511532026 gui = new BubbleSortGUI_2511532026();
220             gui.setVisible(true);
221         });
222     }

```

Output:



2.2 Shell Sort

Shell Sort merupakan pengembangan dari Insertion Sort yang menggunakan konsep jarak (gap) untuk mempercepat proses pengurutan. Data yang berjauhan dibandingkan terlebih dahulu, kemudian nilai gap diperkecil hingga menjadi 1.

Pada program ShellSort_2511532026, nilai gap awal ditentukan sebesar setengah dari panjang array. Setelah setiap iterasi, gap dibagi dua hingga bernilai nol.

Kelebihan:

Lebih cepat dibandingkan Bubble Sort dan Insertion Sort.
Implementasi relatif sederhana.

Kode Program:

```
1 package pekan8_2511532026;
2 public class ShellSort_2511532026 {
3
4     public static void shellSort_2026(int[] A_2026) {
5         int n_2026 = A_2026.length;
6         int gap_2026 = n_2026 / 2;
7
8         while (gap_2026 > 0) {
9             for (int i_2026 = gap_2026; i_2026 < n_2026; i_2026++) {
10
11                 int temp_2026 = A_2026[i_2026];
12                 int j_2026 = i_2026;
13
14                 while (j_2026 >= gap_2026 &&
15                     A_2026[j_2026 - gap_2026] > temp_2026) {
16
17                     A_2026[j_2026] = A_2026[j_2026 - gap_2026];
18                     j_2026 = j_2026 - gap_2026;
19                 }
20
21                 A_2026[j_2026] = temp_2026;
22             }
23
24             gap_2026 = gap_2026 / 2;
25         }
26     }
27
28     public static void main(String[] args_2026) {
29
30         int[] data_2026 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
31
32         System.out.print("Sebelum: ");
33         printArray_2026(data_2026);
34
35         shellSort_2026(data_2026);
36
37         System.out.print("Sesudah (Shell Sort): ");
38         printArray_2026(data_2026);
39     }
40
41     public static void printArray_2026(int[] arr_2026) {
42         for (int i_2026 : arr_2026) {
43             System.out.print(i_2026 + " ");
44         }
45         System.out.println();
46     }
47 }
```

Output:

```
<terminated> ShellSort_2511532026 [Java Application] C:\
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

2.3 Merge Sort

Merge Sort menggunakan metode **Divide and Conquer**, yaitu membagi array menjadi beberapa bagian kecil, mengurutkan setiap bagian, kemudian menggabungkannya kembali menjadi array yang terurut.

Pada program `MergeSort_2511532026`, array dibagi secara rekursif hingga tersisa satu elemen. Setelah itu dilakukan proses *merge* untuk menggabungkan kembali data yang telah terurut.

Kelebihan:

1. Kompleksitas waktu stabil $O(n \log n)$.
2. Cocok untuk data berukuran besar.

Kode Program:

```
1 package pekan8_2511532026;
2 public class MergeSort_2511532026 {
3
4     void merge_2026(int arr_2026[], int l_2026, int m_2026, int r_2026) {
5
6         // Find sizes of two subarrays to be merged
7         int n1_2026 = m_2026 - l_2026 + 1;
8         int n2_2026 = r_2026 - m_2026;
9
10        // Create temp arrays
11        int L_2026[] = new int[n1_2026];
12        int R_2026[] = new int[n2_2026];
13
14        // Copy data to temp arrays
15        for (int i_2026 = 0; i_2026 < n1_2026; ++i_2026)
16            L_2026[i_2026] = arr_2026[l_2026 + i_2026];
17
18        for (int j_2026 = 0; j_2026 < n2_2026; ++j_2026)
19            R_2026[j_2026] = arr_2026[m_2026 + 1 + j_2026];
20
21        // Initial indexes of merged subarray array
22        int i_2026 = 0;
23        int j_2026 = 0;
24
25        int k_2026 = l_2026;
26
27        while (i_2026 < n1_2026 && j_2026 < n2_2026) {
28
29            if (L_2026[i_2026] <= R_2026[j_2026]) {
30                arr_2026[k_2026] = L_2026[i_2026];
31                i_2026++;
32            } else {
33                arr_2026[k_2026] = R_2026[j_2026];
34                j_2026++;
35            }
36
37            k_2026++;
38        }
39
40        // Copy remaining elements of L[] if any
41        while (i_2026 < n1_2026) {
42            arr_2026[k_2026] = L_2026[i_2026];
43            i_2026++;
44            k_2026++;
45        }
46
47        while (j_2026 < n2_2026) {
48            arr_2026[k_2026] = R_2026[j_2026];
49            j_2026++;
50            k_2026++;
51        }
52    }
53
54    void sort_2026(int arr_2026[], int l_2026, int r_2026) {
55
56        if (l_2026 < r_2026) {
57
58            // Find the middle point
59            int m_2026 = (l_2026 + r_2026) / 2;
60
61            // Sort first and second halves
62            sort_2026(arr_2026, l_2026, m_2026);
63            sort_2026(arr_2026, m_2026 + 1, r_2026);
64
65            // Merge the sorted halves
66            merge_2026(arr_2026, l_2026, m_2026, r_2026);
67        }
68    }
69
70    // A utility function to print array of size n
71    static void printArray_2026(int arr_2026[]) {
72
73        int n_2026 = arr_2026.length;
74
75        for (int i_2026 = 0; i_2026 < n_2026; ++i_2026)
76            System.out.print(arr_2026[i_2026] + " ");
77
78        System.out.println();
79    }
80
81    public static void main(String args_2026[]) {
82
83        int arr_2026[] = {12, 11, 13, 5, 6, 7};
84
85        System.out.println("Sebelum terurut");
86        printArray_2026(arr_2026);
87
88        MergeSort_2511532026 ob_2026 = new MergeSort_2511532026();
89
90        ob_2026.sort_2026(arr_2026, 0, arr_2026.length - 1);
91
92        System.out.println("\nSesudah Terurut menggunakan Merge Sort");
93    }
94 }
```

Output:

```
<terminated> MergeSort_2511532026 [Java Application]
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan Merge Sort
5 6 7 11 12 13
```

2.4 Quick Sort

Quick Sort juga menggunakan metode **Divide and Conquer**. Algoritma ini memilih sebuah elemen sebagai pivot, kemudian membagi data menjadi dua bagian, yaitu data yang lebih kecil dan lebih besar dari pivot.

Pada program **QuickSort_2511532026**, pemilihan pivot menggunakan teknik **Median of Three**, yaitu memilih nilai tengah dari elemen pertama, tengah, dan terakhir untuk meningkatkan efisiensi pengurutan.

Kelebihan:

1. Sangat cepat untuk sebagian besar kasus.
2. Tidak membutuhkan memori tambahan yang besar.

Kekurangan:

Pada kondisi tertentu dapat memiliki performa $O(n^2)$.

Kode Program:

```
1 package pekan8_2511532026;
2 public class QuickSort_2511532026 {
3
4     static void swap_2026(int[] arr_2026, int i_2026, int j_2026) {
5         int temp_2026 = arr_2026[i_2026];
6         arr_2026[i_2026] = arr_2026[j_2026];
7         arr_2026[j_2026] = temp_2026;
8     }
9
10    // Memilih pivot menggunakan Median-of-Three
11    static void medianOfThree_2026(int[] arr_2026, int low_2026, int high_2026) {
12
13        int mid_2026 = low_2026 + (high_2026 - low_2026) / 2;
14
15        // Urutkan elemen low, mid dan high
16        if (arr_2026[low_2026] > arr_2026[mid_2026]) {
17            swap_2026(arr_2026, low_2026, mid_2026);
18        }
19        if (arr_2026[low_2026] > arr_2026[high_2026]) {
20            swap_2026(arr_2026, low_2026, high_2026);
21        }
22        if (arr_2026[mid_2026] > arr_2026[high_2026]) {
23            swap_2026(arr_2026, mid_2026, high_2026);
24        }
25        swap_2026(arr_2026, mid_2026, high_2026);
26    }
27
28    static int partition_2026(int[] arr_2026, int low_2026, int high_2026) {
29
30        // Panggil metode medianOfThree sebelum menentukan pivot
31        medianOfThree_2026(arr_2026, low_2026, high_2026);
32
33        int pivot_2026 = arr_2026[high_2026];
34        int i_2026 = (low_2026 - 1);
35
36        for (int j_2026 = low_2026; j_2026 <= high_2026 - 1; j_2026++) {
37
38            // Jika elemen lebih kecil dari atau sama dengan pivot
39            if (arr_2026[j_2026] <= pivot_2026) {
40
41                // Increment indeks elemen yang lebih kecil
42                i_2026++;
43                swap_2026(arr_2026, i_2026, j_2026);
44            }
45        }
46
47        swap_2026(arr_2026, i_2026 + 1, high_2026);
48        return (i_2026 + 1);
49    }
50    static void quickSort_2026(int[] arr_2026, int low_2026, int high_2026) {
51
52        if (low_2026 < high_2026) {
53
54            int pi_2026 = partition_2026(arr_2026, low_2026, high_2026);
55
56            quickSort_2026(arr_2026, low_2026, pi_2026 - 1);
57            quickSort_2026(arr_2026, pi_2026 + 1, high_2026);
58        }
59    }
60
61    public static void printArr_2026(int[] arr_2026) {
62
63        for (int i_2026 = 0; i_2026 < arr_2026.length; i_2026++) {
64            System.out.print(arr_2026[i_2026] + " ");
65        }
66
67        System.out.println();
68    }
69
70    public static void main(String[] args_2026) {
71
72        int[] arr_2026 = {10, 7, 8, 9, 1, 5};
73        int N_2026 = arr_2026.length;
74
75        System.out.print("Data sebelum diurutkan: ");
76        printArr_2026(arr_2026);
77
78        quickSort_2026(arr_2026, 0, N_2026 - 1);
79
80        System.out.print("Data Terurut quicksort: ");
81        printArr_2026(arr_2026);
82    }
83 }
```

Output:

```
<terminated> QuickSort_2511532026 [Java Applicat
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```


BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting memiliki cara kerja dan tingkat efisiensi yang berbeda-beda. Bubble Sort merupakan metode paling sederhana dan mudah dipahami, sedangkan Shell Sort memberikan peningkatan performa dengan penggunaan gap. Merge Sort dan Quick Sort menggunakan pendekatan Divide and Conquer yang lebih efisien untuk data berukuran besar. Implementasi GUI pada Bubble Sort membantu memahami proses pengurutan secara visual sehingga memudahkan pembelajaran algoritma sorting. Secara keseluruhan, pemilihan algoritma sorting harus disesuaikan dengan kebutuhan dan jumlah data yang akan diolah.

3.2 Saran

Dalam pengembangan program sorting selanjutnya, dapat ditambahkan fitur visualisasi untuk algoritma Shell Sort, Merge Sort, dan Quick Sort agar proses pengurutan lebih mudah dipahami. Selain itu, pengujian dengan jumlah data yang lebih besar dapat dilakukan untuk membandingkan performa masing-masing algoritma secara lebih mendalam.

TUGAS PEKAN 8

Kelas Lagu_2511532026

Kelas Lagu_2511532026 digunakan untuk menyimpan data lagu yang akan diurutkan. Setiap objek lagu memiliki tiga atribut, yaitu:

1. judul_2026 : menyimpan judul lagu.
2. penyanyi_2026 : menyimpan nama penyanyi.
3. durasi_2026 : menyimpan lama lagu dalam satuan detik.

Constructor pada kelas ini berfungsi untuk mengisi nilai ketiga atribut tersebut saat objek lagu dibuat.

Kode Program:

```
1 package pekan8_2511532026;
2
3 public class Lagu_2511532026 {
4     String judul_2026;
5     String penyanyi_2026;
6     int durasi_2026;
7
8     public Lagu_2511532026(String judul_2026, String penyanyi_2026, int durasi_2026) {
9         this.judul_2026 = judul_2026;
10        this.penyanyi_2026 = penyanyi_2026;
11        this.durasi_2026 = durasi_2026;
12    }
13 }
```

Kelas Sorting_2511532026

Kode Program:

```
1 package pekan8_2511532026;
2
3 public class Sorting_2511532026 {
4
5     Lagu_2511532026[] dataLagu_2026 = new Lagu_2511532026[20];
6     int jumlahData_2026 = 0;
7
8     public void inputData_2026() {
9         dataLagu_2026[jumlahData_2026++] = new Lagu_2511532026("Hati-Hati di Jalan", "Tulus", 240);
10        dataLagu_2026[jumlahData_2026++] = new Lagu_2511532026("Komang", "Raim Laode", 210);
11        dataLagu_2026[jumlahData_2026++] = new Lagu_2511532026("Sial", "Mahalini", 230);
12        dataLagu_2026[jumlahData_2026++] = new Lagu_2511532026("Melukis Senja", "Budi Doremi", 250);
13        dataLagu_2026[jumlahData_2026++] = new Lagu_2511532026("Monokrom", "Tulus", 220);
14        dataLagu_2026[jumlahData_2026++] = new Lagu_2511532026("Evaluasi", "Hindia", 260);
15        dataLagu_2026[jumlahData_2026++] = new Lagu_2511532026("Rumah ke Rumah", "Hindia", 280);
16    }
17
18    public void tampilkanData_2026() {
19        for (int i = 0; i < jumlahData_2026; i++) {
20            System.out.println((i + 1) + ". " +
21                dataLagu_2026[i].judul_2026 + " - " +
22                dataLagu_2026[i].durasi_2026 + " detik");
23        }
24    }
25
26    public int partition_2026(int low_2026, int high_2026) {
27        int pivot_2026 = dataLagu_2026[high_2026].durasi_2026;
28        int i_2026 = low_2026 - 1;
29
30        for (int j_2026 = low_2026; j_2026 < high_2026; j_2026++) {
31            if (dataLagu_2026[j_2026].durasi_2026 < pivot_2026) {
32                i_2026++;
33
34                Lagu_2511532026 temp_2026 = dataLagu_2026[i_2026];
35                dataLagu_2026[i_2026] = dataLagu_2026[j_2026];
36                dataLagu_2026[j_2026] = temp_2026;
37            }
38        }
39
40        Lagu_2511532026 temp_2026 = dataLagu_2026[i_2026 + 1];
41        dataLagu_2026[i_2026 + 1] = dataLagu_2026[high_2026];
42        dataLagu_2026[high_2026] = temp_2026;
43
44        return i_2026 + 1;
45    }
46 }
```

```

47 public void quickSort_2026(int low_2026, int high_2026) {
48     if (low_2026 < high_2026) {
49         int pi_2026 = partition_2026(low_2026, high_2026);
50
51         quickSort_2026(low_2026, pi_2026 - 1);
52         quickSort_2026(pi_2026 + 1, high_2026);
53     }
54 }
55
56 public static void main(String[] args) {
57
58     Sorting_2511532026 playlist_2026 = new Sorting_2511532026();
59
60     playlist_2026.inputData_2026();
61
62     System.out.println("=== Sorting Playlist NIM: 2511532026 ===");
63     System.out.println("\nData Sebelum Sorting:");
64     playlist_2026.tampilData_2026();
65
66     playlist_2026.quickSort_2026(0, playlist_2026.jumlahData_2026 - 1);
67
68     System.out.println("\nData Setelah Quick Sort (Durasi Asc):");
69     playlist_2026.tampilData_2026();
70 }
71 }

```

Kelas `Sorting_2511532026` merupakan kelas utama yang digunakan untuk menyimpan data playlist lagu dan menjalankan proses pengurutan menggunakan algoritma Quick Sort.

a. Variabel `dataLagu_2026`

```
Lagu_2511532026[] dataLagu_2026 = new Lagu_2511532026[20];
```

Variabel ini berupa array yang digunakan untuk menyimpan maksimal 20 data lagu.

b. Variabel `jumlahData_2026`

```
int jumlahData_2026 = 0;
```

Variabel ini digunakan untuk menghitung jumlah lagu yang telah dimasukkan ke dalam array.

3. Method `inputData_2026()`

Method ini berfungsi untuk mengisi data awal playlist lagu. Pada program ini terdapat tujuh data lagu yang dimasukkan ke dalam array `dataLagu_2026`.

```
public void inputData_2026()
```

Data yang dimasukkan meliputi judul lagu, nama penyanyi, dan durasi lagu.

4. Method `tampilData_2026()`

Method ini digunakan untuk menampilkan seluruh data lagu yang tersimpan dalam array.

```
public void tampilData_2026()
```

Method melakukan perulangan dari indeks pertama hingga jumlah data terakhir, kemudian menampilkan nomor urut, judul lagu, dan durasi lagu.

5. Method partition_2026()

Method partition_2026() merupakan bagian penting dari algoritma Quick Sort.

```
public int partition_2026(int low_2026, int high_2026)
```

Fungsinya adalah memilih elemen terakhir sebagai pivot, kemudian membandingkan seluruh elemen lain dengan pivot tersebut. Data yang memiliki durasi lebih kecil dari pivot akan ditempatkan di sebelah kiri, sedangkan data yang lebih besar akan berada di sebelah kanan. Setelah proses selesai, method mengembalikan posisi pivot yang baru.

6. Method quickSort_2026()

Method ini digunakan untuk melakukan proses pengurutan data menggunakan algoritma Quick Sort.

```
public void quickSort_2026(int low_2026, int high_2026)
```

Method bekerja secara rekursif dengan langkah-langkah berikut:

1. Menentukan posisi pivot menggunakan method partition_2026().
2. Mengurutkan bagian kiri pivot.
3. Mengurutkan bagian kanan pivot.
4. Proses dilakukan berulang sampai seluruh data terurut berdasarkan durasi secara ascending (kecil ke besar).

Kompleksitas rata-rata Quick Sort adalah:

$$O(n \log n)$$

Sedangkan kompleksitas terburuknya adalah:

$$O(n^2)$$

7. Method main()

Method main() merupakan titik awal eksekusi program.

```
public static void main(String[] args)
```

Langkah-langkah yang dilakukan yaitu:

1. Membuat objek Sorting_2511532026.
2. Mengisi data lagu menggunakan inputData_2026().
3. Menampilkan data sebelum sorting.
4. Menjalankan quickSort_2026().
5. Menampilkan data setelah sorting.

Dengan demikian, pengguna dapat melihat perubahan urutan data lagu sebelum dan sesudah proses pengurutan dilakukan.

Output:

```
<terminated> Sorting_2511532026 [Java Application]

Data Sebelum Sorting:
1. Hati-Hati di Jalan - 240 detik
2. Komang - 210 detik
3. Sial - 230 detik
4. Melukis Senja - 250 detik
5. Monokrom - 220 detik
6. Evaluasi - 260 detik
7. Rumah ke Rumah - 200 detik

Data Setelah Quick Sort (Durasi Asc):
1. Rumah ke Rumah - 200 detik
2. Komang - 210 detik
3. Monokrom - 220 detik
4. Sial - 230 detik
5. Hati-Hati di Jalan - 240 detik
6. Melukis Senja - 250 detik
7. Evaluasi - 260 detik
```

